

Task 2: Build an S3 Storage and Backup Workflow

Intern: Nidhi Pandey

Program: Evolixa AWS Cloud Internship

Date: 2026-08-30

1. Approach and Architecture

When I started this task, I imagined a small company that has a lot of application files and needs to keep them safe, organized, and secure. They don't want everything to be public, but they also want to host a simple static website. So, my main approach was to build a well-structured S3 bucket that could double as a file storage solution and a web hosting platform.

I planned to organize files using folders (or prefixes, as AWS calls them) like documents/, backups/, and the website root. I wanted to test both the AWS Console and the CLI to get hands-on experience. The architecture is pretty straightforward—just a single S3 bucket with versioning turned on and a strict bucket policy that allows public read access only for the website files and blocks everything else. Security was the main focus, so I decided to go with the principle of "least privilege" as much as possible.

2. Implementation Steps

Here is how I executed the task, step-by-step:

1. **Bucket Creation:** I created a new bucket named task2-nidhi-website in the ap-south-1 (Mumbai) region. I made sure the name was unique and easy to identify for the internship tasks.
2. **Configuration of Secure Access:** Initially, I kept "Block Public Access" turned on. For the static website, I had to switch it off for

that specific bucket, but I compensated by adding a very strict bucket policy.

3. **Creating Logical Folders:** Inside the bucket, I created a few logical folders like documents/ and backups/ to categorize files properly.
4. **Uploading Sample Files:** I uploaded a few dummy text files and a simple index.html file to test the structure.
5. **Download & Verify:** I downloaded one of the files to my local machine to check if the content matched the original.
6. **Updating a File:** I made a change to the index.html file (added "Version 2.0") and uploaded it again.
7. **Exploring Metadata:** I checked the "Properties" tab of a file to see things like Content-Type, Last-Modified, and ETag.
8. **Enabling Versioning:** I turned on bucket versioning so that I could keep track of all versions of a file. I tested this by uploading multiple versions of a test.txt file.
9. **Testing Private Access:** I temporarily blocked all public access to the bucket to see what would happen. The website returned an "Access Denied" error, which confirmed that the block settings were working.
10. **Using AWS CLI:** I used command-line commands like `aws s3 cp`, `aws s3 ls`, and `aws s3 sync` to handle the files instead of using the console.
11. **Documenting Concepts:** I wrote down the difference between a bucket and an object to make sure I understood the basics.
12. **Understanding Public Risks:** I noted down why having a public bucket is risky if not properly configured.
13. **Storage Structure:** I documented how I organized the folders.
14. **File Types:** I identified that sensitive stuff like credentials should never go into a public bucket.

15. **Cleanup:** I deleted the test files to avoid unnecessary costs.
-

3. Key AWS Concepts Learned

This task helped me really understand how S3 works in a practical setting:

- **Bucket vs Object:** A bucket is just a container, while the object is the actual file along with its metadata.
 - **Bucket Policies:** These JSON files are powerful. They give you fine-grained control over who can do what inside a bucket.
 - **Static Website Hosting:** S3 is not just for backups—it can actually serve a functional website without needing an EC2 server.
 - **Versioning:** It's like a time machine for files. If I mess up a file, I can just restore the old version.
 - **AWS CLI Profiles:** I learned that the CLI works based on the profile you are using. If you use the wrong profile, you get Access Denied errors, even if the bucket exists.
-

4. Challenges Faced

I hit a few roadblocks while working on this task, mainly revolving around permissions:

1. **AWS CLI Permission Issues:** When I tried to run `aws s3 cp`, I kept getting "AccessDenied" errors. I was confused because I knew the bucket existed, and I was using the correct account.
2. **Confusion with "Block Public Access":** I wasn't sure if I should allow public access or rely just on the bucket policy. It took me a bit to understand that the "Block Public Access" switch is a super-admin override that blocks everything, even if the policy says "Allow".
3. **Script Path Errors:** In my local terminal, I wasn't sure if I was in the right directory, and I forgot to use the `--recursive` flag while trying to copy a folder.

5. Solutions Implemented

Here's how I fixed these issues:

1. **CLI Fix:** I realized that my default CLI profile was `sns-user`, which doesn't have any S3 permissions. I started appending `--profile s3-admin` to all my AWS CLI commands. For the Python script, I explicitly used `boto3.Session(profile_name='s3-admin')` to make sure the code used the right credentials.
2. **Public Access Fix:** I learned that "Block Public Access" is the main switch. I turned it off only for this specific static website use case but made the bucket policy extremely restrictive (only allowing `s3:GetObject`).
3. **Syntax Fix:** I double-checked my command syntax and used the proper flags (`--recursive`) to upload directories successfully.

6. Security Considerations

Security was definitely the biggest focus of this task.

- **Least Privilege:** The bucket policy only allows `s3:GetObject`. Even though the bucket is "public" for the website, no one can delete, edit, or upload anything unless they are authenticated with high-level admin credentials.
- **No Hardcoded Secrets:** I made sure not to paste any Access Keys or Secret Keys into the scripts. Everything relies on the CLI profiles, which are stored securely locally.
- **Versioning:** This protects against ransomware or accidental deletions. If a file is overwritten, I can just go back to the previous version.
- **Encryption:** I ensured that the default server-side encryption (SSE-S3) was enabled so data is encrypted at rest.

7. Results and Observations

- **Website Accessible:** The static website is live and accessible via the S3 endpoint URL.
- **Versioning Works:** I tested uploading a file twice, and I could see both versions listed in the console.
- **CLI Automation:** Commands like `aws s3 sync` are incredibly fast. I can sync entire folders in a few seconds, which saves a lot of time compared to clicking through the console.
- **Security Layers:** I confirmed that "Block Public Access" overrides all bucket policies. Once I turned it on, the website stopped working immediately, giving an "Access Denied" message.

I noticed that while the console is good for one-off tasks, the CLI is much more efficient for repetitive or batch operations.

8. Possible Improvements

If this was a real production environment, I would implement the following improvements:

- **CloudFront + OAI:** I would add CloudFront in front of the bucket to improve load times and security. Using an Origin Access Identity (OAI), I can make the S3 bucket completely private (blocking all public access) while still allowing CloudFront to fetch the objects.
- **Lifecycle Policies:** I would set up lifecycle rules to automatically transition older versions of backups to cheaper storage tiers (like S3 Glacier) after a set period to reduce storage costs.
- **MFA Delete:** I would enable MFA Delete to ensure that deleting a whole bucket or a version requires an extra layer of authentication.
- **Custom Domain & SSL:** I would attach a custom domain with an SSL certificate to make the website look more professional and serve content over HTTPS.
- **Monitoring & Logging:** I would enable S3 server access logging and CloudWatch metrics to monitor who is accessing the data and to get alerts for unusual activity.

